Recherche informée

CSI 4506 Introduction à l'intelligence artificielle

Marcel Turcotte
Version: nov. 3, 2026 20h25

Préambule

Message du jour

Objectifs d'apprentissage

- **Distinguer** l'estimation heuristique h, le coût du chemin g et la priorité de recherche f.
- **Implémenter** A* sur graphe à l'aide d'une file de priorité et des meilleurs coûts de chemin connus.
- **Expliquer** l'optimalité en coût sous l'hypothèse d'admissibilité et selon la politique précisée de traitement des chemins vers un même état.
- **Construire et justifier** les heuristiques des tuiles mal placées et de la distance de Manhattan pour le 8-Puzzle.
- **Comparer** l'effort de recherche et les besoins en mémoire dans des conditions contrôlées.

Fonctions Python de la présentation 16

In [1]:
```python
import random
from collections import deque

import matplotlib.pyplot as plt
import numpy as np

random.seed(58)

def is_solvable(tiles):
    # Compter les inversions dans la liste à plat des tuiles (en excluant la
    # Hypothèse : le plateau est carré et sa largeur est impaire.
    inversions = 0
    for i in range(len(tiles)):
        for j in range(i + 1, len(tiles)):
            if tiles[i] != 0 and tiles[j] != 0 and tiles[i] > tiles[j]:
                inversions += 1
    return inversions % 2 == 0
```

```python
def generate_solvable_board():
    # Générer une configuration de plateau aléatoire qui est garantie d'être
    tiles = list(range(9))
    random.shuffle(tiles)
    while not is_solvable(tiles):
        random.shuffle(tiles)

    return tuple(tiles)

def plot_board(board, title, num_pos, position):
    ax = plt.subplot(1, num_pos, position)
    ax.set_title(title)
    ax.set_xticks([])
    ax.set_yticks([])

    board = np.array(board).reshape(3, 3).tolist() # Reconfigurer en une gri

    # Utiliser une carte de couleurs pour afficher les numéros
    cmap = plt.cm.plasma
    norm = plt.Normalize(vmin=-1, vmax=8)

    for i in range(3):
        for j in range(3):
            tile_value = board[i][j]
            color = cmap(norm(tile_value))
            ax.add_patch(plt.Rectangle((j, 2 - i), 1, 1, facecolor=color, ec
            if tile_value == 0:
                ax.add_patch(plt.Rectangle((j, 2 - i), 1, 1, facecolor='whit
            else:
                ax.text(j + 0.5, 2 - i + 0.5, str(tile_value),
                        fontsize=16, ha='center', va='center', color='black'

    ax.set_xlim(0, 3)
    ax.set_ylim(0, 3)

def is_empty(frontier):
    """Vérifie si la frontière est vide."""
    return len(frontier) == 0

def is_goal(state, goal_state):
    """Détermine si un état donné correspond à l'état objectif."""
    return state == goal_state

def expand(state):
    """Génère les états successeurs en déplaçant la case vide dans toutes le
    size = int(len(state) ** 0.5) # Déterminer la taille du puzzle (3 pour l
    idx = state.index(0) # Trouver l'index de la case vide représentée par 0
    x, y = idx % size, idx // size # Convertir l'index en coordonnées (x, y)
    neighbors = []

    # Définir les mouvements possibles : Gauche, Droite, Haut, Bas
    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        # Vérifier si la nouvelle position est dans les limites du puzzle
```

```
        if 0 <= nx < size and 0 <= ny < size:
            n_idx = ny * size + nx
            new_state = list(state)
            # Échanger la case vide avec la tuile adjacente
            new_state[idx], new_state[n_idx] = new_state[n_idx], new_state[i
            neighbors.append(tuple(new_state))
    return neighbors

def print_solution(solution):
    """Affiche la séquence d'étapes de l'état initial à l'état objectif."""
    size = int(len(solution[0]) ** 0.5)
    for step, state in enumerate(solution):
        print(f"Étape {step} :")
        for i in range(size):
            row = state[i*size:(i+1)*size]
            print(' '.join(str(n) if n != 0 else ' ' for n in row))
        print()

def bfs(initial_state, goal_state):
    frontier = deque() # Initialiser la file d'attente pour BFS
    frontier.append((initial_state, [])) # Un nœud : (état, états précédents

    reached = {initial_state}

    expanded = 0 # Nombre de nœuds non objectifs dont les successeurs sont g

    while not is_empty(frontier):
        current_state, path = frontier.popleft()

        if is_goal(current_state, goal_state):
            print(f"Nœuds développés : {expanded}")
            return path + [current_state] # Retourner le chemin réussi

        expanded += 1

        for neighbor in expand(current_state):
            if neighbor not in reached:
                reached.add(neighbor)
                frontier.append((neighbor, path + [current_state]))

    return None

def dfs(initial_state, goal_state):
    frontier = [(initial_state, [])] # Un nœud : (état, états précédents)

    reached = {initial_state}

    expanded = 0

    while not is_empty(frontier):
        current_state, path = frontier.pop()

        if is_goal(current_state, goal_state):
            print(f"Nœuds développés : {expanded}")
            return path + [current_state] # Retourner le chemin réussi
```

```
        expanded += 1

        for neighbor in expand(current_state):
            if neighbor not in reached:
                reached.add(neighbor)
                frontier.append((neighbor, path + [current_state]))

    return None
```

Ces fonctions ont été introduites dans la présentation 16 et sont nécessaires pour exécuter les exemples de la présentation 17. Affichez le code pour revoir la représentation des états, la génération des successeurs, les tests d'objectif et les méthodes BFS et DFS qui servent de référence.

La formulation d'un problème, le vocabulaire des arbres de recherche, la frontière et les méthodes BFS et DFS sont présentés dans la présentation 16. Les fonctions regroupées ici permettent d'exécuter le carnet de manière autonome et fournissent une référence pour revoir ces concepts.

Un état de recherche est un tuple de neuf entiers à une dimension, où `0` représente la case vide. Les tuples d'entiers sont immuables et hachables ; les états peuvent donc servir directement d'éléments d'un ensemble ou de clés d'un dictionnaire. Seules la génération des plateaux et la construction des successeurs utilisent des listes modifiables temporaires ; chaque état ainsi construit est retourné sous forme de tuple. Les conteneurs de la frontière et les chemins restent distincts de la représentation d'un état. Un ensemble d'états atteints conserve les états découverts, y compris ceux qui se trouvent encore dans la frontière.

Remarques

- La **recherche en largeur (BFS)** trouve une solution optimale de 25 déplacements après le développement de 145 605 nœuds non objectifs.
- La **recherche en profondeur (DFS)** trouve une solution de 1 157 déplacements après le développement de 1 187 nœuds non objectifs.

Comment trouver des solutions **plus efficacement** ?

Recherche informée

Recherche heuristique

Une **recherche informée** utilise une estimation du **coût restant jusqu'à un état objectif** pour guider l'exploration.

. . .

Comment une telle estimation pourrait-elle aider à **choisir le prochain nœud de la frontière** ?

L'estimation utilise des connaissances qui s'ajoutent aux actions, au modèle de transition, au test d'objectif et aux coûts d'action du problème. Un état objectif peut être une ville, un puzzle résolu ou une condition sur un état abstrait ; il n'a pas nécessairement une position physique.

L'estimation guide le choix des possibilités à examiner. Elle ne remplace pas le modèle de transition et ne vérifie pas qu'une solution complète existe.

Heuristiques et priorités de recherche

Pour un nœud de recherche n représentant l'état s :

- $g(n)$: **coût connu de ce chemin** de l'état initial à s.
- $h(s)$: **estimation du coût restant minimal** de s à un état objectif.
- $f(n)$: **priorité** utilisée pour sélectionner une entrée de la frontière.

Stratégie par meilleur choix	**Priorité $f(n)$**
Recherche gloutonne par meilleur choix	$h(s)$
Recherche à coût uniforme	$g(n)$
Recherche A*	$g(n) + h(s)$

La **recherche par meilleur choix** sélectionne une entrée de la frontière dont f est minimal. L'heuristique h et le coût de chemin g doivent utiliser les mêmes unités de coût.

Un état décrit une configuration, tandis qu'un nœud de recherche conserve aussi un chemin pour l'atteindre. Deux nœuds peuvent donc avoir le même état et la même estimation heuristique, mais des coûts de chemin différents. La notation $h(n)$ sert souvent de raccourci pour $h(s)$, où s est l'état conservé dans n.

La recherche gloutonne utilise uniquement le coût restant estimé et n'offre aucune garantie générale de coût minimal. A* considère aussi le coût déjà engagé. La recherche

à coût uniforme est le cas particulier d'A* où $h = 0$. Voir Berkeley CS 188, Recherche informée.

Une heuristique pour la recherche d'itinéraires

Dans un problème de distance routière, la **distance à vol d'oiseau** jusqu'à la destination estime la distance restante.

. . .

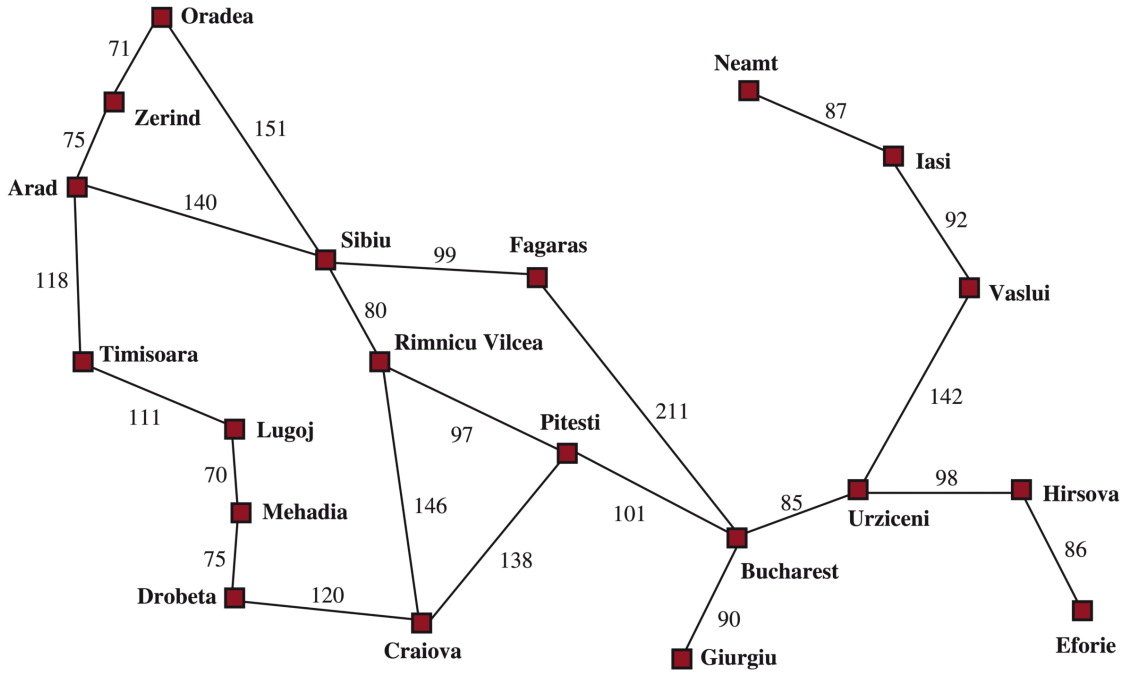
La ligne droite n'a **pas** à correspondre à une route utilisable. A* ajoute cette estimation à la distance déjà parcourue :

$$f(n) = g(n) + h(s).$$

Les deux termes sont mesurés en kilomètres dans l'exemple suivant. La distance à vol d'oiseau est une borne inférieure de la distance routière. Elle n'estimerait pas directement le temps de trajet sans une conversion appropriée en unités de temps.

L'exemple de la Roumanie utilise A* : les priorités affichées incluent la distance parcourue et la distance à vol d'oiseau. Sélectionner uniquement la ville ayant la plus petite distance à vol d'oiseau correspondrait plutôt à la recherche gloutonne par meilleur choix.

Roumanie : problème de recherche d'itinéraires



Source : (Russell et Norvig 2020, fig. 3.1)

Problème : Trouver un itinéraire de distance routière minimale d'Arad à Bucarest. Les états sont des villes, les actions suivent les routes et les coûts d'action sont les distances routières en kilomètres.

Roumanie : heuristique de distance à vol d'oiseau

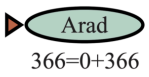
Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

Source : (Russell et Norvig 2020, fig. 3.16)

Le tableau donne les distances à vol d'oiseau jusqu'à Bucarest. Ce sont les valeurs heuristiques $h(s)$, et non les coûts totaux des itinéraires.

Roumanie : frontière initiale

(a) The initial state

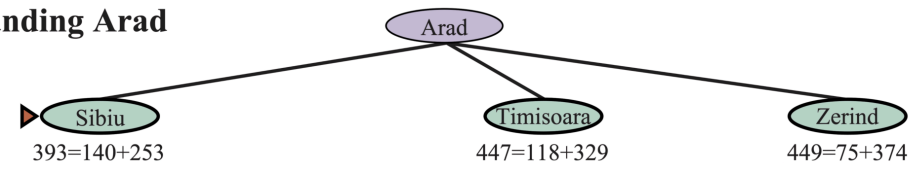


Source : (Russell et Norvig 2020, fig. 3.18)

Arad entre dans la frontière avec $g(n) = 0$ et $h(s) = 366$; sa priorité est donc $f(n) = 366$. Chaque total affiché dans les figures suivantes est une valeur de $f = g + h$.

Roumanie : développement d'Arad

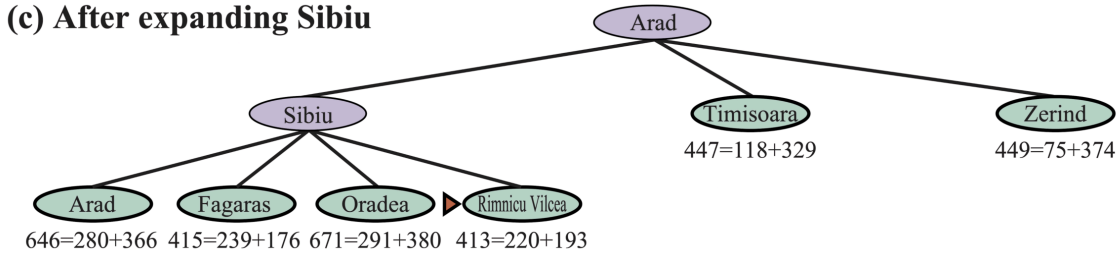
(b) After expanding Arad



Source : (Russell et Norvig 2020, fig. 3.18)

Depuis Arad, les nouvelles entrées sont Sibiu ($140 + 253 = 393$), Timisoara ($118 + 329 = 447$) et Zerind ($75 + 374 = 449$). Sibiu a le plus petit f et est sélectionnée ensuite.

Roumanie : développement de Sibiu

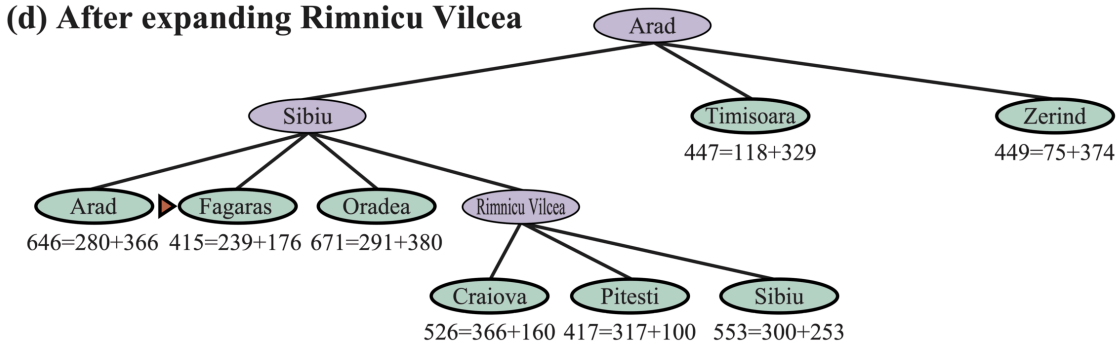


Source : (Russell et Norvig 2020, fig. 3.18)

Le développement de Sibiu génère des chemins vers Arad, Fagaras, Oradea et Rimnicu Vilcea. Pour Rimnicu Vilcea, $g = 220$, $h = 193$ et $f = 413$; elle est donc sélectionnée avant Fagaras, dont la priorité est 415.

Ces figures conservent les occurrences de l'arbre de recherche, y compris les villes répétées. L'implémentation de recherche sur graphe développée ci-dessous écarte plutôt les chemins dominés et conserve une nouvelle entrée lorsque le coût enregistré pour une ville s'améliore.

Roumanie : développement de Rimnicu Vilcea



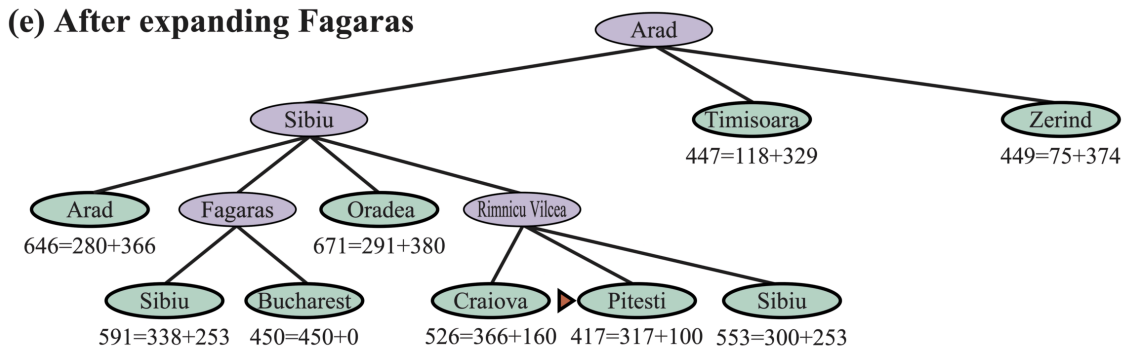
Source : (Russell et Norvig 2020, fig. 3.18)

Depuis Rimnicu Vilcea, les candidates sont Craiova ($366 + 160 = 526$), Pitesti ($317 + 100 = 417$) et Sibiu ($300 + 253 = 553$). La deuxième occurrence de Sibiu représente un chemin plus coûteux vers la même ville ; le coût de son premier chemin

était de 140. Une implémentation de recherche sur graphe écarterait l'itinéraire de coût 300 vers Sibiu.

Quelle entrée de la frontière est sélectionnée ensuite ? Fagaras a encore la plus petite priorité, $f = 415$, suivie de Pitesti à 417.

Roumanie : développement de Fagaras



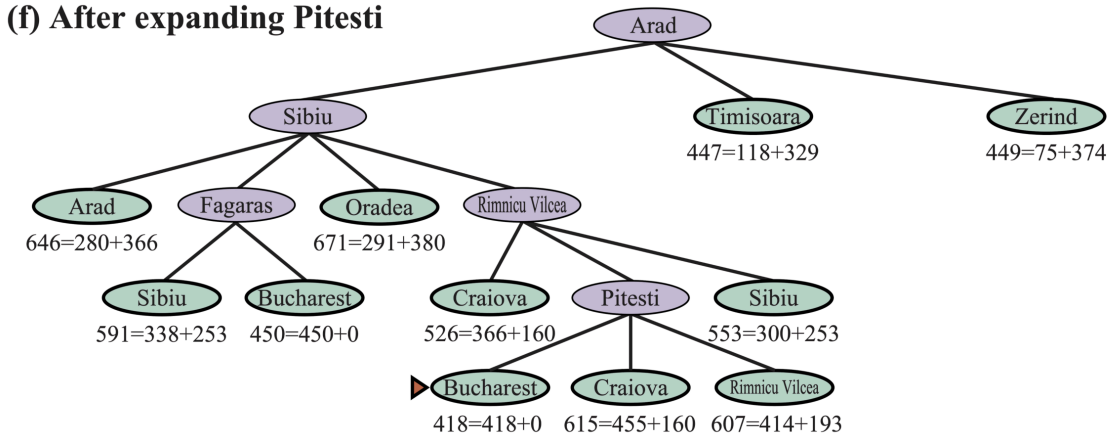
Source : (Russell et Norvig 2020, fig. 3.18)

Le développement de Fagaras génère une entrée pour Bucarest avec $g = 450$ et $h = 0$, donc $f = 450$. Il s'agit d'un itinéraire complet, mais générer une entrée correspondant à un état objectif ne termine pas A*.

Pitesti ($f = 417$), Timisoara (447) et Zerind (449) ont toutes des valeurs de priorité inférieures à celle de cette entrée objectif. Pitesti est sélectionnée ensuite.

Le test d'objectif est appliqué lorsqu'une entrée à jour de valeur f minimale est retirée de la frontière. À un état objectif, $h = 0$ et $f = g$; la recherche ne rend pas intrinsèquement l'heuristique plus précise au fil de l'exécution.

Roumanie : sélection de Bucarest



Source : (Russell et Norvig 2020, fig. 3.18)

Le développement de Pitesti génère une entrée moins coûteuse pour Bucarest, de coût 418. Puisque $h(\mathrm{Bucharest}) = 0$, sa priorité est aussi 418, inférieure à celles des autres entrées de la frontière.

L'itinéraire retourné est Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucarest, pour une distance routière totale de **418 km**. L'entrée précédente pour Bucarest, de coût 450, illustre pourquoi un état déjà atteint peut nécessiter un chemin moins coûteux.

Pourquoi la sélection de cet état objectif suffit-elle à établir l'optimalité en coût ? L'implémentation et la preuve suivantes précisent les conditions requises.

Une file de priorité

Comment la frontière de BFS/DFS doit-elle changer lorsque les nœuds ont des priorités différentes ?

. . .

Utiliser une **file de priorité** ordonnée selon la fonction d'évaluation f.

In [2]:
```
import heapq
```

Le module `heapq` de Python retire la plus petite entrée. Pour A*, la première composante est $g + h$.

BFS comme recherche par meilleur choix

BFS sélectionne les nœuds par **profondeur** non décroissante. On peut la décrire à l'aide de la fonction d'évaluation

$$f(n) = \mathrm{depth}(n).$$

La profondeur est connue à partir du chemin ; ce n'est pas une estimation heuristique du coût restant jusqu'à l'objectif. Lorsque les coûts d'action sont égaux, la profondeur et le coût du chemin induisent le même ordre entre les niveaux. Une file FIFO est une implémentation de BFS plus simple qu'un tas et conserve son ordre habituel à l'intérieur d'un niveau.

On peut décrire DFS de manière semblable en donnant la priorité aux nœuds plus profonds, à l'aide de la profondeur négative et d'une règle appropriée de départage « dernier entré, premier sorti ». Le choix de la règle de départage compte lorsque plusieurs entrées ont la même priorité.

Recherche A*

$$f(n) = g(n) + h(s).$$

- $g(n)$ est le coût du **chemin particulier** conservé dans l'entrée n.
- $h(s)$ estime le **coût restant minimal** depuis son état s.
- A* sélectionne une entrée de la frontière dont f **est minimal**.

Hart et al. (1968)

Connaître exactement $g(n)$ pour un chemin enregistré ne signifie pas qu'il s'agit du chemin le moins coûteux vers cet état. Un autre itinéraire peut l'améliorer. Le dictionnaire de l'implémentation suivante conserve le plus petit coût découvert jusqu'à présent.

L'implémentation du 8-Puzzle utilise des coûts d'action unitaires ; prolonger un chemin fait donc passer g à $g + 1$. La même idée s'applique à d'autres modèles de coût en ajoutant le coût réel de l'action.

Implémentation d'A*

In [3]:
```python
def astar(initial_state, goal_state, heuristic):
    # Les entrées contiennent (f, g, état, états précédents).
    frontier = [(heuristic(initial_state, goal_state), 0, initial_state, [])
    best_g = {initial_state: 0}
    expanded = 0

    while frontier:
        f, g, current_state, path = heapq.heappop(frontier)
        # Un chemin moins coûteux peut avoir remplacé cette entrée de la fil
        if g > best_g[current_state]:
            continue

        if is_goal(current_state, goal_state):
            return path + [current_state], expanded

        expanded += 1
        for neighbor in expand(current_state):
            new_g = g + 1  # Chaque déplacement du puzzle coûte 1.
            if neighbor not in best_g or new_g < best_g[neighbor]:
                best_g[neighbor] = new_g
                new_f = new_g + heuristic(neighbor, goal_state)
                heapq.heappush(frontier, (new_f, new_g, neighbor, path + [cu

    return None, expanded
```

La fonction retourne `(solution_states, expanded)`. Une solution inclut les états initial et objectif ; son nombre de déplacements est donc `len(solution_states) - 1`.

Conserver les chemins de moindre coût

- `best_g[state]` conserve le **plus petit coût de chemin découvert jusqu'à présent**, et non un coût définitif.
- Ajouter un successeur à la file lorsque son nouveau coût de chemin est **strictement inférieur**.
- Ignorer une entrée retirée si son `g` enregistré est supérieur à `best_g[state]`.
- Un chemin moins coûteux peut provoquer la **réouverture** d'un état déjà développé.
- Appliquer le test d'objectif uniquement après avoir retiré une **entrée à jour de valeur f minimale**.

Une entrée de la frontière représente un chemin particulier vers un état. Plusieurs entrées peuvent désigner le même état, avec des coûts de chemin différents. Lorsqu'un meilleur chemin est trouvé, la nouvelle entrée est ajoutée ; une entrée plus ancienne et plus coûteuse peut rester dans le tas. Le test des entrées obsolètes l'écarte avant le test d'objectif ou le développement. Par exemple, si un état est placé dans la file à un coût de 7, puis atteint à un coût de 5, `best_g` devient 5 ; l'ancienne entrée de coût 7 est ignorée lorsqu'elle est retirée.

Un état développé n'est pas définitivement fermé dans cette implémentation. Si son coût enregistré s'améliore, ses successeurs doivent être considérés à nouveau avec ce chemin moins coûteux. Le résultat `expanded` compte les développements réels de nœuds non objectifs, y compris les développements répétés ; il ne compte ni les entrées obsolètes ni l'état objectif sélectionné.

Pour le 8-Puzzle, chaque action coûte un ; le nouveau coût de chemin est donc `g + 1`. Lorsque les coûts d'action diffèrent, la même règle utilise plutôt `g + action_cost`.

La mise à jour du meilleur coût suit la règle présentée dans AIMA, Figure 3.7.

Admissibilité

Une heuristique est **admissible** si elle ne surestime jamais le coût restant minimal d'une solution :

$$0 \leq h(s) \leq h^\star(s).$$

Ici, $h^\star(s)$ est le véritable coût minimal de s à un état objectif.

Que pourrait-il se passer si l'estimation surestimait le coût restant ?

Une surestimation peut faire paraître peu attrayante une solution réellement peu coûteuse ; A* peut alors retourner une solution plus coûteuse. Une surestimation ne force pas une mauvaise réponse dans chaque cas ; elle supprime la garantie générale.

À tout état objectif G, l'admissibilité donne $h(G) = 0$, car le coût restant est nul. Cette présentation utilise des valeurs heuristiques non négatives. Si aucun état objectif n'est accessible depuis un état, son véritable coût restant est infini.

La condition sur l'heuristique ne constitue pas à elle seule tout l'algorithme : l'implémentation doit aussi conserver les chemins améliorés et appliquer le test d'objectif au bon moment.

Optimalité en coût

Un algorithme est **optimal en coût** lorsqu'il retourne une solution de coût de chemin minimal chaque fois qu'il retourne une solution.

. . .

Pour la recherche A* sur graphe ci-dessus, la garantie combine :

- une **heuristique admissible** ;
- les **mises à jour du meilleur coût et la réouverture** ;
- un test d'objectif sur une **entrée à jour de valeur f minimale**.

La preuve concerne la même implémentation que celle utilisée dans les exemples et les expériences. Le test des entrées obsolètes garantit qu'une entrée dépassée n'est pas traitée comme le meilleur itinéraire actuel vers son état. La complétude pose en plus la question de savoir si l'algorithme trouve une solution lorsqu'il en existe une ; les espaces finis et infinis exigent des hypothèses de terminaison différentes.

Théorème : optimalité en coût

Considérons le **graphe fini des états accessibles** de l'implémentation ci-dessus, avec des **coûts d'action unitaires**. Soit $h^\star(s)$ le coût restant minimal de l'état s à un état objectif.

L'heuristique est **admissible** :

$$0 \le h(s) \le h^\star(s), \qquad h(G) = 0 \text{ pour tout état objectif } G.$$

Avec les **règles de meilleur coût et de réouverture ci-dessus**, A* retourne une **solution de coût minimal** lorsqu'il en existe une et signale un échec sinon.

Le 8-Puzzle, fini et à coûts unitaires, satisfait ces hypothèses. Le résultat s'étend aussi aux graphes finis dont les coûts d'action sont non négatifs, lorsque l'implémentation ajoute le coût réel de chaque action. Si aucun état objectif n'est accessible depuis un état, son coût restant jusqu'à un état objectif est infini ; l'admissibilité n'y impose aucune

borne supérieure finie. L'heuristique utilisée par l'implémentation est une estimation fixe et non négative pour chaque état.

Les règles de gestion des chemins comptent autant que l'admissibilité. Écarter tout chemin ultérieur vers un état simplement parce qu'il a été généré plus tôt ne satisfait pas ce théorème.

Preuve

1. Supposons qu'A* sélectionne un état objectif sous-optimal.

Soit $C^\star$ le coût de la solution optimale. Supposons que l'algorithme retourne une entrée objectif G de coût $C > C^\star$.

Puisque $h(G) = 0$,

$$f(G) = g(G) + h(G) = C.$$

Examinons la frontière immédiatement avant la sélection de cette entrée objectif à jour.

Preuve

2. Une continuation de coût optimal reste dans la frontière.

Suivons un chemin optimal depuis le départ. Considérons son premier état qui **n'a pas encore été développé au coût optimal de son préfixe**.

Une **entrée à jour de la frontière** atteint cet état avec ce coût optimal de préfixe.

L'état initial entre dans la frontière avec un coût nul. Chaque fois qu'un état du chemin optimal choisi est développé au coût optimal de son préfixe, la génération de l'action suivante propose à l'état suivant son propre coût optimal de préfixe.

Si cela améliore `best_g`, la nouvelle entrée est ajoutée à la file, même si l'état a été développé plus tôt par un chemin plus coûteux. Si le même coût optimal était déjà enregistré, son entrée attend encore ou a déjà été développée à ce coût. Dans le second cas, continuons le long du chemin. Aucun préfixe moins coûteux ne peut exister, car remplacer ce préfixe produirait une solution moins coûteuse.

Le premier état qui n'a pas encore été développé au coût optimal de son préfixe a donc une entrée en attente à ce coût. Elle n'est pas obsolète

> `best_g` ne peut pas devenir inférieur au coût optimal du préfixe. Un tel état doit exister, à moins qu'un état objectif optimal ait déjà été sélectionné ; dans ce cas, l'algorithme aurait déjà retourné une solution.

Un développement antérieur à un coût plus élevé n'invalide pas cet argument : la règle d'amélioration stricte provoque la réouverture de l'état lorsque son préfixe optimal est découvert.

Preuve

3. La borne inférieure de cette entrée de la frontière ne dépasse pas $C^\star$.

Notons n cette entrée et s son état. Son $g(n)$ enregistré est le coût optimal du préfixe. Par admissibilité,

$$f(n) = g(n) + h(s) \leq g(n) + h^\star(s) = C^\star.$$

Le préfixe et la continuation restante de coût minimal forment ensemble une solution optimale.

Preuve

4. La règle du f minimal contredit la sélection de l'état objectif sous-optimal.

La frontière contient une entrée à jour n telle que

$$f(n) \leq C^\star < C = f(G).$$

A* doit sélectionner une entrée dont f est inférieur avant G. Il ne peut donc pas retourner un état objectif sous-optimal.

Sur un graphe fini, les cycles de coût non négatif ne peuvent pas améliorer le coût d'un chemin. Accepter uniquement les améliorations strictes empêche d'ajouter indéfiniment des cycles de coût égal ; seul un nombre fini de chemins améliorés peut être accepté. La recherche se termine donc. Si elle épuise la frontière, il ne reste aucune solution à trouver.

Sur un graphe **infini**, l'admissibilité seule ne garantit pas la complétude. Un ensemble classique d'hypothèses suffisantes est un branchement fini, l'existence d'une solution de coût fini et des coûts d'action bornés inférieurement par une valeur $\varepsilon > 0$. Puisque $h \geq 0$, seul un nombre fini de chemins candidats peut avoir $f \leq C^\star$ sous ces hypothèses. Cela exclut une succession infinie d'étapes de coût nul qui retarderait indéfiniment une solution. Une recherche dans un espace infini sans solution ne termine pas nécessairement en signalant un échec.

Cohérence (consistency)

Une heuristique est **cohérente** si chaque action de s à s' satisfait

$$h(s) \le c(s, s') + h(s'), \qquad h(G) = 0.$$

Son coût restant estimé ne peut pas diminuer de plus que le coût de cette action. Par conséquent, le long d'un chemin,

$$f(n') = g(n) + c(s, s') + h(s') \ge g(n) + h(s) = f(n).$$

La cohérence est une condition locale sur chaque transition. L'appliquer successivement le long d'un chemin vers un état objectif montre que l'heuristique est admissible. Dans le 8-Puzzle, un déplacement de coût unitaire peut réduire le nombre de tuiles mal placées d'au plus un et peut réduire la distance de Manhattan d'au plus un ; les deux heuristiques sont cohérentes.

Avec une heuristique cohérente, la première sélection d'une entrée à jour de valeur f minimale pour un état utilise déjà un chemin de coût minimal vers cet état. La réouverture après le développement est donc inutile. La même implémentation d'`astar` fonctionne toujours ; elle n'a simplement pas besoin de développer à nouveau un état à un meilleur coût.

La cohérence ne justifie pas de considérer comme définitif le coût d'un état dès sa première génération : un chemin moins coûteux peut encore être trouvé avant son premier développement.

Une heuristique admissible mais incohérente peut nécessiter une réouverture. La règle du meilleur coût préserve l'optimalité dans ce cas, même si les développements répétés peuvent augmenter le travail. Une variante de recherche sur graphe qui ferme définitivement les états après leur développement s'appuie plutôt sur la cohérence pour cette garantie. Voir Berkeley CS 188, Recherche informée.

Construire une heuristique pour le 8-Puzzle

Quelles informations sur un plateau permettraient d'estimer le nombre de déplacements restants ?

Chaque action coûte une unité ; l'heuristique doit donc estimer un nombre de déplacements. Les exemples excluent la case vide du décompte des tuiles mal placées et de la somme des distances des tuiles numérotées. Leur objectif est le plateau ordonné standard, avec la case vide dans le coin inférieur droit.

Le nombre d'inversions peut-il servir d'heuristique ?

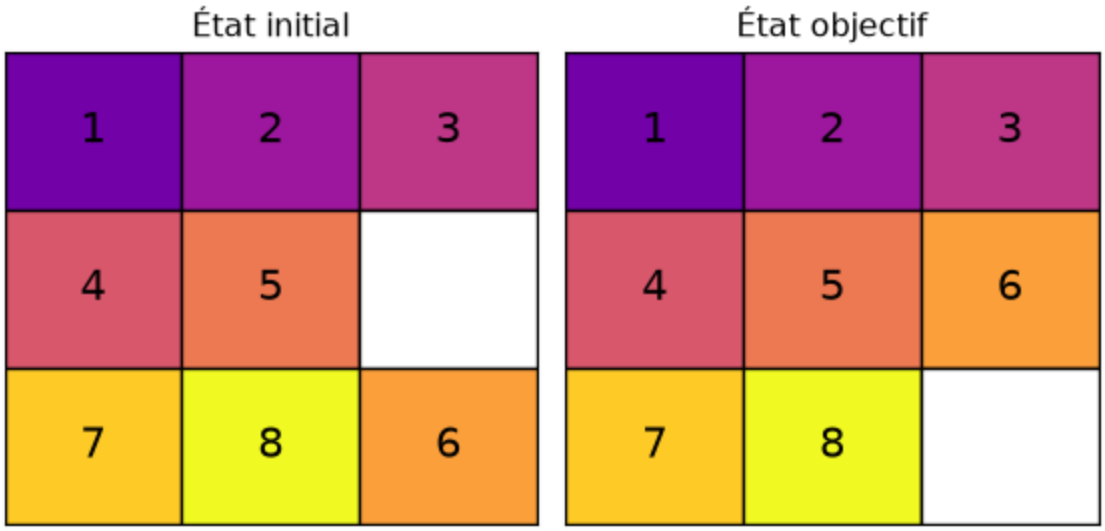
Considérons le **nombre brut d'inversions** comme $h(s)$.

. . .

In [4]:
```python
# Définir l'état initial
initial_board = (1, 2, 3,
                 4, 5, 0,
                 7, 8, 6)

# Définir l'état objectif
goal_board = (1, 2, 3,
              4, 5, 6,
              7, 8, 0)

# Afficher les deux plateaux
plt.figure(figsize=(6, 3))
plot_board(initial_board, "État initial", 2, 1)
plot_board(goal_board, "État objectif", 2, 2)
plt.tight_layout()
plt.show()
```



- Calculez $h(s)$ et $h^\star(s)$ pour ce plateau.
- Que permet d'établir cette comparaison ?

Le plateau présenté contient deux inversions : la tuile 6 vient après les tuiles 7 et 8. Déplacer 6 dans la case vide résout le puzzle en un déplacement. Ainsi, $h(s) = 2 > 1 = h^\star(s)$, ce qui réfute l'admissibilité du nombre brut d'inversions. La même transition vers un état objectif viole la cohérence : $2 > 1 + 0$.

Sur un **plateau 3×3**, un déplacement permis fait varier le nombre d'inversions de 0 ou de ±2. Un déplacement vertical peut faire passer une tuile numérotée devant deux autres tuiles numérotées dans l'ordre de lecture par rangées. Le contre-exemple montre pourquoi le nombre brut d'inversions n'est pas une borne inférieure du nombre de déplacements ; la variation possible de ±2 ne suffit pas, à elle seule, à prouver la non-admissibilité.

La parité remplit toutefois une autre fonction. Le plateau `(1,2,3,4,5,6,0,8,7)` a une parité d'inversion impaire et ne peut pas atteindre l'état objectif de parité paire. La

résolubilité et le coût restant d'une solution sont deux questions différentes.

Tuiles mal placées

In [5]:
```
def misplaced_tiles_distance(state, goal_state):

    # Compter le nombre de tuiles mal placées

    misplaced_tiles = sum(1 for s, g in zip(state, goal_state) if s != g and

    return misplaced_tiles
```

Cette heuristique est-elle admissible ? La **case vide est exclue**.

Un déplacement permis change la position d'exactement une tuile numérotée. Il peut corriger au plus une tuile mal placée ; les autres tuiles numérotées conservent leur position. Si $h(s)$ tuiles sont mal placées, toute solution exige donc au moins $h(s)$ déplacements, d'où $h(s) \leq h^\star(s)$.

La même observation établit la cohérence : un déplacement peut réduire le nombre de tuiles mal placées d'au plus une unité, donc $h(s) \leq 1 + h(s')$. La valeur de l'état objectif est nulle.

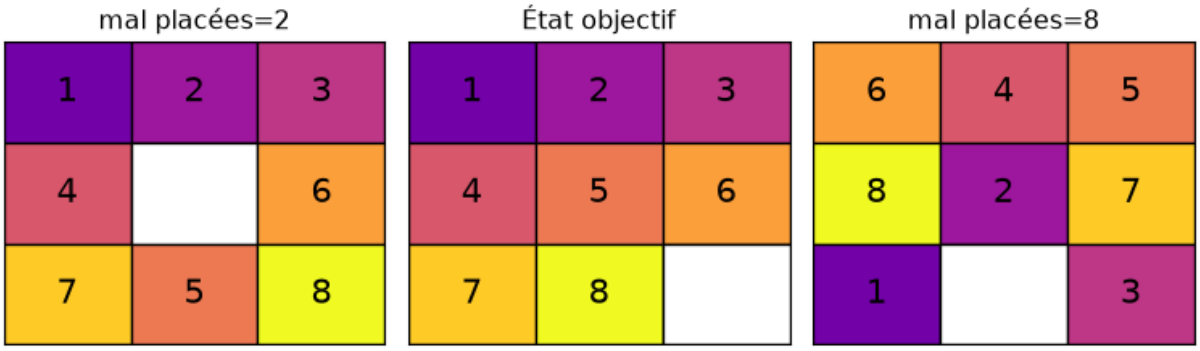
Tuiles mal placées sur deux plateaux

In [6]:
```
GOAL_STATE = (1, 2, 3,
              4, 5, 6,
              7, 8, 0)
SIMPLE_STATE = (1, 2, 3,
                4, 0, 6,
                7, 5, 8)
HARD_STATE = (6, 4, 5,
              8, 2, 7,
              1, 0, 3)

def plot_heuristic_comparison(left, right, goal, heuristics):
    plt.figure(figsize=(8, 2.5))
    for position, state in [(1, left), (3, right)]:
        title = "\n".join(
            f"{name}={heuristic(state, goal)}"
            for name, heuristic in heuristics.items()
        )
        plot_board(state, title, 3, position)
    plot_board(goal, "État objectif", 3, 2)
    plt.tight_layout()
    plt.show()

plot_heuristic_comparison(
    SIMPLE_STATE, HARD_STATE, GOAL_STATE,
```

```
    {"mal placées": misplaced_tiles_distance},
)
```



Appeler A* avec une heuristique

```
solution_states, expanded = astar(
    SIMPLE_STATE, GOAL_STATE, misplaced_tiles_distance
)
```

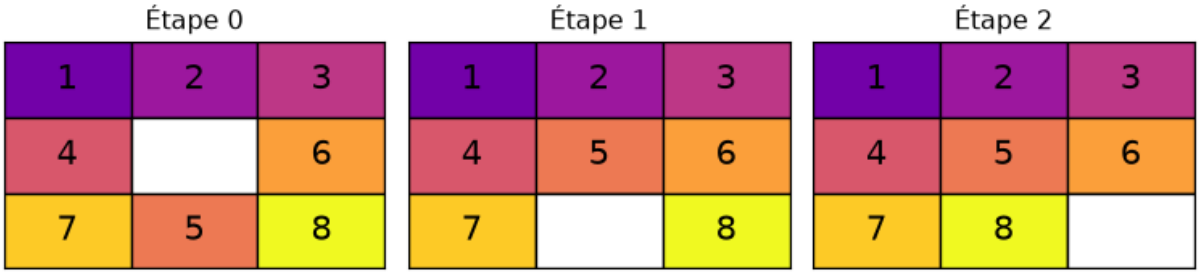
Une seule implémentation de recherche accepte l'heuristique comme argument. Elle retourne une séquence d'états et le nombre de développements de nœuds non objectifs.

Cas simple : tuiles mal placées

In [7]:
```
solution_states, expanded = astar(SIMPLE_STATE, GOAL_STATE, misplaced_tiles_
print(f"Solution : {len(solution_states) - 1} déplacements ; nœuds développé

plt.figure(figsize=(8, 2))
for i, state in enumerate(solution_states):
    plot_board(state, f"Étape {i}", len(solution_states), i + 1)
plt.tight_layout()
plt.show()
```

```
Solution : 2 déplacements ; nœuds développés : 2
```



Cas difficile : tuiles mal placées

In [8]:
```
misplaced_solution, misplaced_expanded = astar(
    HARD_STATE, GOAL_STATE, misplaced_tiles_distance
)
```

```
print(f"Solution : {len(misplaced_solution) - 1} déplacements")
print(f"Nœuds développés : {misplaced_expanded:,}")
```

```
Solution : 25 déplacements
Nœuds développés : 29,005
```

Le chemin retourné comprend les deux extrémités. Utilisez `print_solution(misplaced_solution)` pour examiner une séquence d'états complète.

Distance de Manhattan

In [9]:
```
def manhattan_distance(state, goal_state):
    distance = 0
    size = int(len(state) ** 0.5)
    for num in range(1, len(state)):
        idx1 = state.index(num)
        idx2 = goal_state.index(num)
        x1, y1 = idx1 % size, idx1 // size
        x2, y2 = idx2 % size, idx2 // size
        distance += abs(x1 - x2) + abs(y1 - y2)
    return distance
```

$$h_{\text{Manhattan}}(s) = \sum_{t=1}^{8} \left(|x_t - x_t^\star| + |y_t - y_t^\star| \right).$$

La case vide est exclue. Pourquoi cette somme est-elle une borne inférieure du nombre de déplacements nécessaires ?

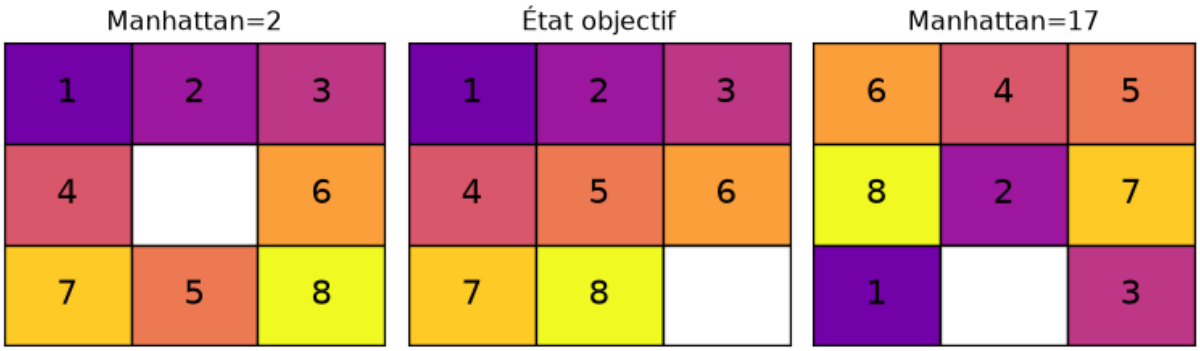
Pour la tuile t, (x_t, y_t) est sa position actuelle et $(x_t^\star, y_t^\star)$ est sa position dans l'état objectif. La somme porte sur les huit tuiles numérotées.

Une action permise déplace une tuile numérotée d'une case, faisant varier sa contribution à la distance de Manhattan d'exactement une unité, à la hausse ou à la baisse. La somme peut diminuer d'au plus une unité par déplacement. Pour atteindre une somme nulle, il faut donc au moins autant de déplacements que la somme initiale des distances de Manhattan, ce qui prouve l'admissibilité. La même observation établit la cohérence : $h(s) \leq 1 + h(s')$.

L'interprétation par un problème relâché rend les tuiles indépendantes : chaque tuile numérotée peut se déplacer horizontalement ou verticalement sans avoir besoin de la case vide, et les contraintes d'occupation sont ignorées. Dans ce problème relâché, chaque tuile peut suivre un plus court chemin sur la grille jusqu'à sa cible ; le coût total exact est donc la somme des distances de Manhattan. Retirer des contraintes ne peut pas augmenter le coût minimal d'une solution, ce qui fournit une borne inférieure pour le puzzle réel.

Valeurs de Manhattan sur deux plateaux

In [10]:
```
plot_heuristic_comparison(
    SIMPLE_STATE, HARD_STATE, GOAL_STATE,
    {"Manhattan": manhattan_distance},
)
```



Comparer les heuristiques

Pour tout état atteignable, en excluant la case vide,

$$h_{\text{misplaced}}(s) \leq h_{\text{Manhattan}}(s) \leq h^{\star}(s).$$

. . .

- Chaque tuile numérotée mal placée contribue au moins une unité à la distance de Manhattan.
- Manhattan **domine** donc le nombre de tuiles mal placées : c'est une borne inférieure plus informative.
- Cela garantit-il un temps d'exécution réel plus court pour chaque instance ?

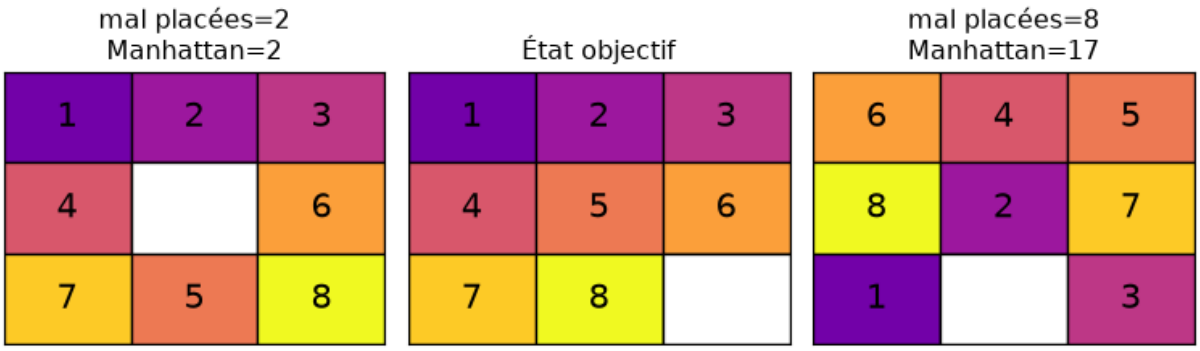
Une estimation admissible plus élevée peut écarter davantage de chemins peu prometteurs. Elle ne produit pas nécessairement strictement moins de développements pour chaque problème. Les entrées dont la priorité égale le coût optimal peuvent être traitées différemment selon les règles de départage, et le calcul d'une heuristique plus informative peut coûter plus cher par nœud.

Les expériences gardent fixes l'algorithme, les instances du problème et la règle de départage, et mesurent les développements de nœuds non objectifs. Elles ne mesurent pas le temps d'exécution écoulé. Voir Berkeley CS 188, Dominance.

Les deux estimations sur les mêmes plateaux

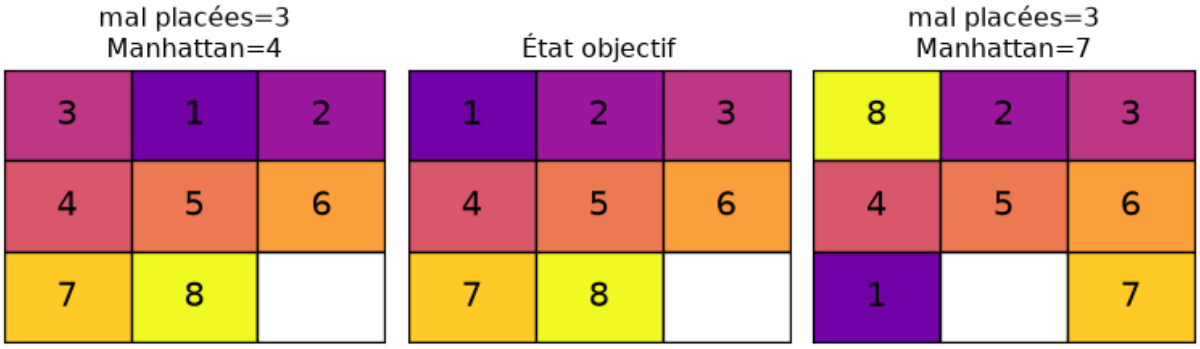
In [11]:
```
plot_heuristic_comparison(
    SIMPLE_STATE, HARD_STATE, GOAL_STATE,
```

```
    {"mal placées": misplaced_tiles_distance, "Manhattan": manhattan_distanc
)
```



Même nombre de tuiles mal placées, distances différentes

In [12]:
```
comparison_left = (3, 1, 2,
                   4, 5, 6,
                   7, 8, 0)
comparison_right = (8, 2, 3,
                    4, 5, 6,
                    1, 0, 7)
plot_heuristic_comparison(
    comparison_left, comparison_right, GOAL_STATE,
    {"mal placées": misplaced_tiles_distance, "Manhattan": manhattan_distanc
)
```



Les deux plateaux ont trois tuiles numérotées mal placées, mais leurs estimations de Manhattan diffèrent. Le nombre de tuiles mal placées indique uniquement si une tuile est à sa position cible ; Manhattan tient aussi compte de la distance qui la sépare de cette position.

Changer l'heuristique

```
solution_states, expanded = astar(
    SIMPLE_STATE, GOAL_STATE, manhattan_distance
)
```

Seul l'argument correspondant à l'heuristique change. La frontière, les mises à jour des meilleurs coûts, le test d'objectif et la reconstruction du chemin restent identiques.

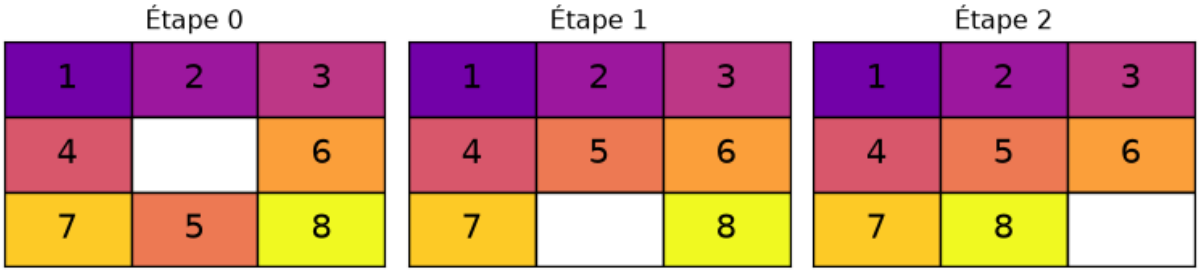
La frontière peut contenir des entrées correspondant à un état objectif. Leur valeur heuristique est nulle ; leur priorité est donc le coût du chemin complet qu'elles représentent. Une autre entrée de priorité inférieure dans la frontière peut encore mener à un objectif moins coûteux. C'est pourquoi l'algorithme applique le test d'objectif après avoir sélectionné une entrée courante de priorité minimale.

Cas simple : Manhattan

In [13]:
```
solution_states, expanded = astar(SIMPLE_STATE, GOAL_STATE, manhattan_distar
print(f"Solution : {len(solution_states) - 1} déplacements ; nœuds développé

plt.figure(figsize=(8, 2))
for i, state in enumerate(solution_states):
    plot_board(state, f"Étape {i}", len(solution_states), i + 1)
plt.tight_layout()
plt.show()
```

```
Solution : 2 déplacements ; nœuds développés : 2
```



Cas difficile : Manhattan

In [14]:
```
manhattan_solution, manhattan_expanded = astar(
    HARD_STATE, GOAL_STATE, manhattan_distance
)
print(f"Solution : {len(manhattan_solution) - 1} déplacements")
print(f"Nœuds développés : {manhattan_expanded:,}")
```

```
Solution : 25 déplacements
Nœuds développés : 2,255
```

Le chemin retourné comprend les deux extrémités. Utilisez `print_solution(manhattan_solution)` pour examiner une séquence d'états complète.

Expériences appariées

- Utilisez les **mêmes 1 000 plateaux résolubles** avec les deux heuristiques.

- Conservez la **même implémentation de A* et la même politique de départage**.
- Conservez les deux longueurs de solution et comparez les **développements de nœuds**.

Les entrées du tas sont `(f, g, state, path)`. Les égalités de `f` sont départagées d'abord par la plus petite valeur de `g`, puis par l'ordre lexicographique du tuple représentant l'état. Cette politique déterministe est commune aux deux exécutions avec les heuristiques; elle peut différer de l'ordre d'une file FIFO au sein d'une même profondeur.

Le générateur aléatoire local rend l'expérience indépendante des tirages aléatoires des exemples précédents. Les deux tableaux de longueurs permettent une vérification directe des résultats sans écraser l'un d'eux. L'accord entre les deux exécutions avec les heuristiques ne constitue pas à lui seul une preuve d'optimalité; une vérification utile de l'implémentation compare aussi un échantillon de taille limitée aux résultats de BFS.

In [15]:
```
experiment_rng = random.Random(58)
experiment_states = []
while len(experiment_states) < 1000:
    state = tuple(experiment_rng.sample(range(9), 9))
    if is_solvable(state):
        experiment_states.append(state)

moves_misplaced = []
moves_manhattan = []
expanded_misplaced = []
expanded_manhattan = []

for initial_state in experiment_states:
    path_misplaced, count_misplaced = astar(
        initial_state, GOAL_STATE, misplaced_tiles_distance
    )
    path_manhattan, count_manhattan = astar(
        initial_state, GOAL_STATE, manhattan_distance
    )
    moves_misplaced.append(len(path_misplaced) - 1)
    moves_manhattan.append(len(path_manhattan) - 1)
    expanded_misplaced.append(count_misplaced)
    expanded_manhattan.append(count_manhattan)

assert moves_misplaced == moves_manhattan
print(f"Plateaux résolubles appariés : {len(experiment_states)}")
print("Les deux heuristiques donnent le même coût de solution pour chaque pl
print(f"Développements moyens, tuiles mal placées : {np.mean(expanded_mispla
print(f"Développements moyens, Manhattan :         {np.mean(expanded_manhatt
```

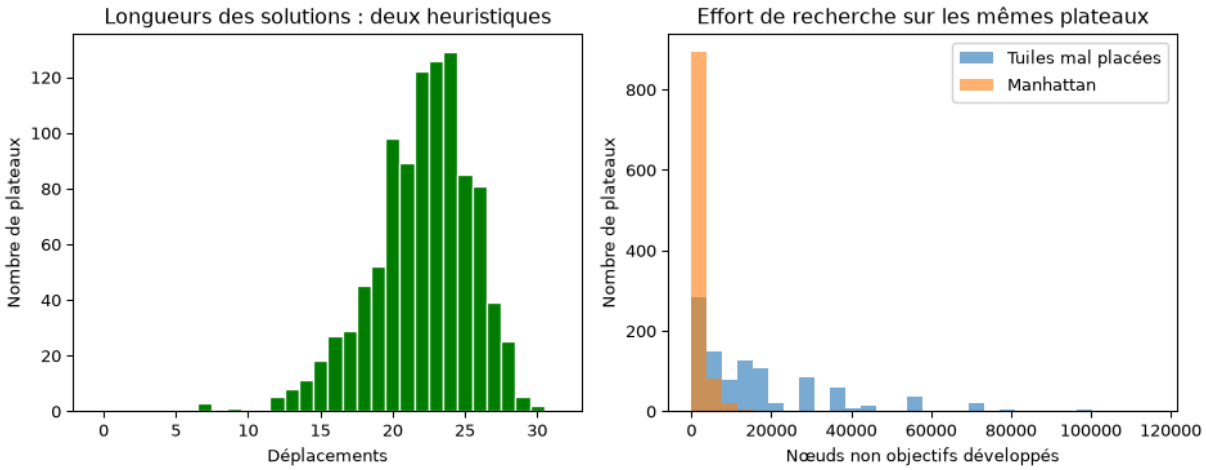
```
Plateaux résolubles appariés : 1000
Les deux heuristiques donnent le même coût de solution pour chaque plateau.
Développements moyens, tuiles mal placées : 16,469.7
Développements moyens, Manhattan :         1,708.2
```

1 000 recherches appariées

In [16]:
```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].hist(moves_misplaced, bins=np.arange(-0.5, 32.5), color='green', edg
axes[0].set_title("Longueurs des solutions : deux heuristiques")
axes[0].set_xlabel("Déplacements")
axes[0].set_ylabel("Nombre de plateaux")

bins = np.histogram_bin_edges(expanded_misplaced + expanded_manhattan, bins=
axes[1].hist(expanded_misplaced, bins=bins, alpha=0.6, label="Tuiles mal pla
axes[1].hist(expanded_manhattan, bins=bins, alpha=0.6, label="Manhattan")
axes[1].set_title("Effort de recherche sur les mêmes plateaux")
axes[1].set_xlabel("Nœuds non objectifs développés")
axes[1].set_ylabel("Nombre de plateaux")
axes[1].legend()
plt.tight_layout()
plt.show()
```

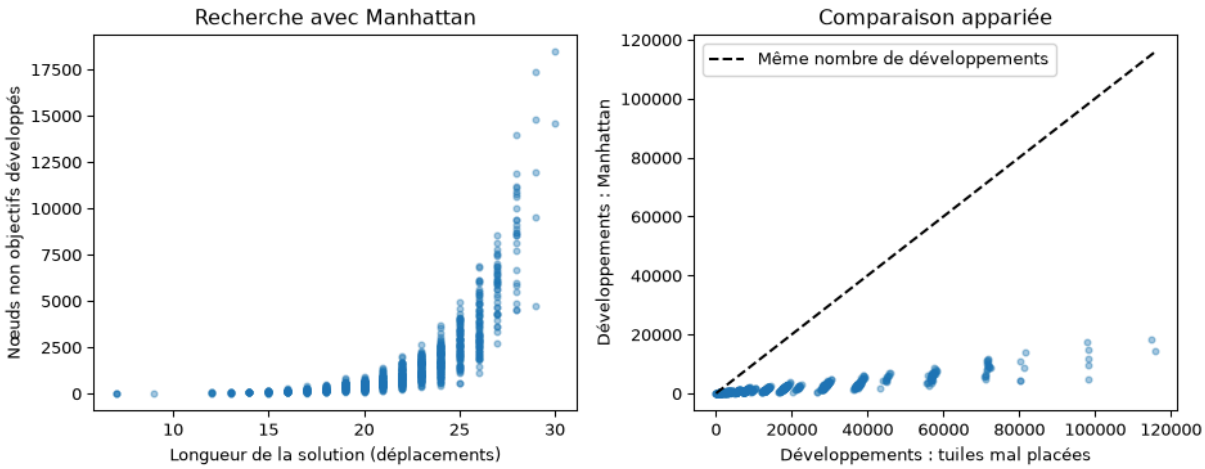


Comparaison de l'effort de recherche

In [17]:
```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].scatter(moves_manhattan, expanded_manhattan, s=12, alpha=0.4)
axes[0].set_title("Recherche avec Manhattan")
axes[0].set_xlabel("Longueur de la solution (déplacements)")
axes[0].set_ylabel("Nœuds non objectifs développés")

axes[1].scatter(expanded_misplaced, expanded_manhattan, s=12, alpha=0.4)
limit = max(max(expanded_misplaced), max(expanded_manhattan))
axes[1].plot([0, limit], [0, limit], '--', color='black', label="Même nombre
axes[1].set_title("Comparaison appariée")
axes[1].set_xlabel("Développements : tuiles mal placées")
axes[1].set_ylabel("Développements : Manhattan")
axes[1].legend()
plt.tight_layout()
plt.show()
```



Chaque point du graphique de droite représente un plateau. Un point sous la diagonale indique moins de développements avec Manhattan pour ce plateau.

Exploration : valider l'optimalité

BFS retourne une solution avec un nombre minimal de déplacements lorsque chaque action coûte un.

1. Générez un échantillon de taille limitée de plateaux résolubles du 8-Puzzle.
2. Trouvez une solution avec BFS et une solution avec A* pour chaque plateau.
3. Vérifiez que chaque chemin est légal, atteint l'état objectif et comporte le même nombre de déplacements.
4. Répétez avec les deux heuristiques admissibles.

BFS fournit une référence indépendante pour le coût de la solution; il n'est pas nécessaire qu'il suive le même chemin que A*. Plusieurs solutions optimales peuvent exister. Limitez la taille de l'échantillon de validation, car BFS peut visiter une grande partie de l'espace d'états accessible pour chaque plateau.

Exploration : une heuristique nulle

Avec $h(s) = 0$, A* ordonne sa frontière uniquement selon $g(n)$: il devient une **recherche à coût uniforme**.

. . .

Lorsque toutes les actions coûtent un, la recherche à coût uniforme et BFS développent les nœuds par **profondeur non décroissante**.

Leur ordre de développement et leur nombre de développements doivent-ils être identiques?

Les deux méthodes donnent le même coût minimal de solution, mais leur ordre au sein d'une même profondeur peut différer. La règle de départage du tas, l'ordre des successeurs, la politique de gestion des doublons et l'emplacement du test d'état objectif influencent l'ordre et le travail effectué avant la sélection de l'état objectif. La recherche dans un graphe évite les états répétés; elle n'énumère pas tous les chemins possibles.

Résultats sur le plateau de la présentation 16

Méthode	Longueur de la solution	Nœuds non objectifs développés
BFS	25 déplacements (optimal)	145 605
DFS	1 157 déplacements	1 187
A* avec Manhattan	25 déplacements (optimal)	2 255

Ces résultats portent sur un plateau, un ordre des successeurs et une politique de départage fixes. Le nombre de développements ne mesure ni le temps d'exécution réel ni la mémoire totale.

Les résultats de référence de BFS et DFS proviennent de la présentation 16. Le résultat de A* utilise l'unique implémentation corrigée ci-dessus. Toutes les méthodes retournent des chemins, mais seuls BFS avec des coûts d'action égaux et A* dans les conditions énoncées garantissent une solution de coût minimal.

Mesurer les performances

- **Complétude :** L'algorithme trouve-t-il une solution dès lors qu'il en existe une?
- **Optimalité en coût :** Retourne-t-il une solution dont le coût du chemin est minimal?

Sur un graphe fini d'états accessibles, les implémentations de BFS et DFS avec détection des états répétés sont complètes. L'implémentation corrigée de A* est également complète sous les hypothèses du théorème. Elles peuvent épuiser la frontière et signaler un échec lorsqu'aucun état objectif n'est accessible.

Dans les espaces infinis, trouver une solution existante et terminer pour tout problème sans solution sont deux exigences différentes. DFS peut suivre une branche infinie. Les hypothèses usuelles de complétude de A* comprennent un facteur de branchement fini, une solution de coût fini et des coûts d'action minorés par une constante strictement positive.

BFS garantit un coût optimal lorsque les coûts d'action sont égaux. DFS n'offre aucune garantie générale d'optimalité en coût. A* repose sur l'admissibilité et sur les politiques précisées de conservation des meilleurs coûts, de réouverture et de sélection d'un état objectif.

Complexité en temps et en espace

- **Graphe de l'espace d'états :** utilisez le nombre d'états V et d'actions E.
- **Arbre de recherche :** utilisez le facteur de branchement b et la profondeur de solution d.
- **Implémentation réelle :** tenez aussi compte des tas, de l'évaluation des heuristiques et des chemins mémorisés.

Les exemples indiquent les **développements de nœuds**. Une comparaison complète des performances doit aussi tenir compte du travail et de la mémoire utilisés par chaque développement.

Avec des opérations sur les états en temps constant et des enregistrements de taille constante par nœud, la recherche BFS ordinaire dans un graphe prend un temps $\mathcal{O}(V + E)$ et un espace $\mathcal{O}(V)$. Le code de la présentation copie des chemins complets dans les entrées de la frontière, ce qui ajoute du travail et de la mémoire proportionnels à leurs longueurs; ces coûts doivent être pris en compte dans la description de l'implémentation.

Pour une recherche BFS sous forme d'arbre avec test d'état objectif à la génération, la borne habituelle est de $\mathcal{O}(b^d)$ nœuds générés. Avec un test au retrait de la frontière, comme dans notre code, des nœuds à la profondeur $d + 1$ peuvent être générés avant qu'un état objectif à la profondeur d ne soit sélectionné. L'analyse correspondante de l'arbre dans le pire cas donne alors $\mathcal{O}(b^{d+1})$ occurrences générées ou mémorisées. Les bornes sur le graphe de l'espace d'états et celles sur l'arbre de recherche comptent des objets différents.

A* ajoute des opérations de file de priorité et des évaluations de l'heuristique. Avec des heuristiques admissibles mais incohérentes, il peut aussi développer à nouveau un état après la découverte d'un chemin amélioré. Le nombre de développements peut donc dépasser le nombre d'états distincts développés. Une petite frontière n'élimine pas la mémoire conservée dans les enregistrements des meilleurs coûts et dans les chemins copiés.

Vidéos de Sebastian Lague

- A* Pathfinding (E01: algorithm explanation) publiée le 2014-12-16.
- A* Pathfinding (E02: node grid) publiée le 2014-12-18.
- A* Pathfinding (E03: algorithm implementation) publiée le 2014-12-19.
- A* Pathfinding (E04: heap optimization) publiée le 2014-12-24.
- A* Pathfinding (E05: units) publiée le 2015-01-06.
- A* Pathfinding (E06: weights) publiée le 2015-01-11.
- A* Pathfinding (E07: smooth weights) publiée le 2016-12-30.
- A* Pathfinding (E08: path smoothing 1/2) publiée le 2017-01-31.

- A* Pathfinding (E09: path smoothing 2/2) publiée le 2017-01-31.
- A* Pathfinding (E10: threading) publiée le 2017-02-03.
- A* Pathfinding Tutorial (Unity) (liste de lecture)

Une ressource consacrée à $A^\star$

- Amit's A* Pages

Épilogue

Résumé

- Les priorités de la recherche par meilleur choix distinguent la **recherche gloutonne**, la **recherche à coût uniforme** et **A***.
- A* combine le **coût connu du chemin** et une **estimation du coût restant**.
- Les **mises à jour des meilleurs coûts, la réouverture et la sélection d'un état objectif** relient l'implémentation à la preuve d'optimalité.
- L'**admissibilité, la cohérence et la dominance** permettent de justifier et de comparer les heuristiques du 8-Puzzle.

Une entrée de la frontière de A* représente un chemin particulier. `best_g` enregistre le plus faible coût découvert vers chaque état, tandis que des entrées obsolètes peuvent rester dans le tas jusqu'à leur retrait. Un chemin amélioré peut rouvrir un état. Ces politiques préservent l'entrée de la frontière sur un chemin optimal utilisée dans la preuve.

Le nombre de tuiles mal placées et la distance de Manhattan excluent la case vide. Les deux heuristiques sont admissibles et cohérentes pour les déplacements de coût un du 8-Puzzle. Manhattan domine le nombre de tuiles mal placées, mais le coût de calcul et les choix de départage comptent toujours dans la mesure des performances.

Sur le plateau fixe de la présentation 16, BFS et A* avec Manhattan retournent tous deux 25 déplacements, avec respectivement 145 605 et 2 255 nœuds non objectifs développés. DFS retourne un chemin plus long après moins de développements. L'expérience appariée compare les deux heuristiques sur des plateaux identiques et conserve séparément les coûts de leurs solutions.

Prochaine séance

- **Quiz 2 :** le 18 novembre 2026. Consultez la FAQ du cours.
- Après le quiz : Recherche locale, présentation 19.

Références

Hart, Peter E., Nils J. Nilsson, et Bertram Raphael. 1968. « A Formal Basis for the Heuristic Determination of Minimum Cost Paths ». *IEEE Transactions on Systems Science and Cybernetics* 4 (2): 100-107. https://doi.org/10.1109/tssc.1968.300136.

Russell, Stuart, et Peter Norvig. 2020. *Artificial Intelligence: A Modern Approach*. 4e éd. Pearson. http://aima.cs.berkeley.edu/.

Marcel **Turcotte**

Marcel.Turcotte@uOttawa.ca

École de génie électrique et d'**informatique** (ÉGÉ**I**)

Université d'Ottawa